IndustEdge: A Time-Sensitive Networking Enabled Edge-Cloud Collaborative Intelligent Platform for Smart Industry

Yimeng Wang, Shusen Yang, *Senior Member, IEEE*, Xuebin Ren, *Member, IEEE*, Peng Zhao, Cong Zhao, and Xinyu Yang

***Abstract*—An edge-cloud collaborative intelligent (ECCI) platform is of great significance for the agile development and rapid deployment of ECCI applications, which are essential for realizing smart industry in the era of Industry 4.0. However, the existing platforms lack considering the high real-time latency demand of industrial operations, which severely hinders the development of smart industry and may even lead to severe industrial accidents. To effectively reduce the response latency of industrial applications, in this article, we propose an ECCI platform IndustEdge. It takes time-sensitive networking as the deterministic transport for the link layer, and provides an extensible ECCI orchestration component to reduce the system level latency. Furthermore, IndustEdge has an ECCI algorithm library for different collaborative modes and provides the complete life cycle management for ECCI applications. We implement platforms for both the real-world prototype and emulated-world emulation, and conduct two case studies to evaluate the effectiveness of IndustEdge.**

***Index Terms*—Edge cloud collaborative intelligence (ECCI), edge computing platform, Industry 4.0, smart industry, time -sensitive networking (TSN).**

Manuscript received March 10, 2021; revised June 1, 2021 and July 10, 2021; accepted August 2, 2021. Date of publication September 8, 2021; date of current version December 27, 2021. This work was supported in part by the National Key Research and Development Program of China under Grant 2017YFB1010004, in part by the National Natural Science Foundation of China under Grant 61772410, Grant 61802298, Grant U1811461, and Grant 11690011, and in part by the China Postdoctoral Science Foundation under Grant 2020T130513 and Grant 2019M663726. Paper no. TII-21-1143. *(Corresponding author: Shusen Yang.)*

Yimeng Wang, Xuebin Ren, and Peng Zhao are with the National Engineering Laboratory for Big Data Analytics and School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an 710049, China, also with Pazhou Laboratory, Guangzhou 510335, China (e-mail: wang.yimeng@stu.xjtu.edu.cn; xuebinren@mail.xjtu.edu.cn; p.zhao@mail.xjtu.edu.cn).

Shusen Yang is with the National Engineering Laboratory for Big Data Analytics and Ministry of Education Key Lab for Intelligent Networks and Network Security, Xi'an Jiaotong University, Xi'an 710049, China, and also with Pazhou Laboratory, Guangzhou 510335, China (e-mail: shusenyang@mail.xjtu.edu.cn).

Cong Zhao is with the Department of Computing, Imperial College London, SW7 2AZ London, U.K. (e-mail: c.zhao@imperial.ac.uk).

Xinyu Yang is with the National Engineering Laboratory for Big Data Analytics and School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: yxyphd@mail.xjtu.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TII.2021.3104003.

Digital Object Identifier 10.1109/TII.2021.3104003

I. Introduction

With the rapid development of cyber-physical systems (CPS), artificial intelligence (AI), Internet of Things (IoT) paradigm, and related technologies, a new industrial revolution of Industry 4.0 (I4.0) is now in progress [1]. The vision of I4.0 is to build smart industry, an industrial internet of things, machines, computers and people enabling intelligent industrial operations using advanced data analytics for the transformation of business outcomes [2]. With this trend, data generated by the heterogeneous smart ends (e.g., sensors and cameras) are growing exponentially, and several shortcomings of the traditional cloud computing paradigm [3], [4] have become emergent, such as the high latency, bandwidth, and cost, but the low availability of the connection [5].

As a proliferating paradigm, edge computing [6] aims at pushing computations from the cloud to network edges for the reduction of response latency, bandwidth occupation, and privacy risks. The core value of edge computing is to realize edge intelligence or edge cloud collaborative intelligence (ECCI) [7], i.e., agile, low-cost, and low-latency AI services and applications through collaborative optimization among the ends (end devices, such as the on-site equipments and sensors), edges (e.g., industrial gateways, Arduino boards, and micro data centers) and the cloud (the cloud server). In recent years, increasing efforts have been made concentrating on the development of ECCI industrial systems [8], where edge devices with heterogeneous resource limitations offload communication intensive services from the powerful but remote cloud and pull computations from resource-constrained ends. With the support of ECCI industrial systems, ECCI applications have begun to show a bright future for smart industry. For example, in smart manufacturing with ECCI, the fault detection service can be modularized, and necessary analysis of on-site data for fault detection can be conducted on edge devices that are closer to the equipment. In such a case, the service response latency could be significantly reduced and more prompt system actuation could be achieved.

However, the end-edge-cloud layered architecture and large-scale distributed and heterogeneous devices in the smart industry bring great challenges to the development and deployment of ECCI applications as follows. 1) How to overcome the obstacles caused by local area network, which prevents the cloud from

1551-3203 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

directly accessing the edge devices, and the poor network link between edge and cloud, to provide the stable and efficient communication for the modules of ECCI applications? 2) How to reduce the latency of information technology (e.g., data sensing analysis) and operation technology (e.g., monitoring and actuation of physical processes) in smart industrial scenarios, to significantly improve labor productivity? The existing mature intelligent platforms based on the cloud [9]–[11] are all designed for distributed computing systems with abundant resources and relative homogeneousness, which cannot be directly applied to the resource-constrained and highly heterogeneous ECCI industrial scenarios. Although some ECCI platforms [12], [13] have been proposed, they are lacking in consideration of the high real-time latency requirement for industrial operations. This seriously hinders the industrial operation and may even lead to severe industrial accidents.

To mitigate the issues above, we propose IndustEdge, which, to the best of our knowledge, is the first ECCI platform integrated with IEEE 802.1 time-sensitive networking (TSN) for smart industry. Considering that TSN can only guarantee network transmission latency, IndustEdge provides an extensible ECCI orchestration component that can further reduce the application latency via deploying the microservices of applications on to different edge/cloud devices. In particular, the unified software development kit (SDK) is supported to develop various orchestration algorithms based on resource constraints of the devices and characteristics of microservices and TSN. In addition, IndustEdge can provide unified access and management of the infrastructure (i.e., all devices at the end, edge, and cloud) and the life cycle management of ECCI applications. Moreover, we provide two case studies (*an intrusion detection system built in the laboratory* and *a predictive maintenance system for cutting tools in the manufacturing workshops of an aviation engine manufactory*) to evaluate the effectiveness of IndustEdge. Furthermore, to solve the high cost of actual deployment in the prototype platform, we also provide the corresponding emulation platform for IndustEdge. The technical contributions of this article can be summarized as follows.

1) We propose IndustEdge, an ECCI platform with significantly less response latency for industrial applications. On the one hand, it takes TSN as the deterministic transport for the link layer. On the other hand, it provides an extensible ECCI orchestration component to reduce the system level latency.
2) Maintained by the image repository, IndustEdge provides users with extensive ECCI algorithms, supporting both different analytic tasks (such as classification, clustering, and correlation analysis) and collaborative modes (including edge-cloud collaboration, edge-edge collaboration, etc.). With a Docker-based operation and maintenance system, IndustEdge allows the full life cycle management of ECCI applications, including development, deployment, management, and monitoring. Besides, the stable and transparent communication between the microservices of applications is realized by automatic routing without users' interference.
3) In addition to the prototype platform, an emulation platform is proposed, which can also emulate the TSN-enabled network topology construction, and the processes and interactions of ECCI applications. In particular, the emulation platform can realize efficient emulation for ECCI applications with ten-microsecond level delay and gigabit bandwidth.

The rest of this article is organized as follows. In Section II, we discuss some related work. In Section III, we present the architecture of the IndustEdge prototype and emulation platforms along with the implementation details. Section IV presents the system level latency analysis. Then, two case studies and the performance evaluation are discussed in detail in Section V. Finally, Section VI concludes this article.

II. Related Work

A. ECCI Platforms

Currently, popular ECCI platforms fall into the following two broad categories. One category focuses on the management, communication, and access of IoT devices, such as Alibaba's Link IoT Edge [13], and Microsoft's Azure IoT Edge [12]. However, they can hardly achieve collaboration among the edge devices. The edge-cloud collaborations on these platforms are also relatively inflexible, requiring complex configurations for applications at both the edge gateway and cloud service center. The other category focuses on the unified management and deployment of computing, storage, and network resources on the edge and cloud data centers to achieve efficient application construction processes. Accordingly, such typical platforms include KubeEdge[1] and Baetyl[2]. However, they do not support transparent application communication to users (i.e., users need to configure the routing of the application) and do not provide the ECCI algorithm library.

Considering the large scale real-world deployment is quite costly, a simulation/emulation framework often can provide a convenient validation for ECCI applications. The existing ECCI simulators are mainly implemented through three models. The most popular platforms are usually built on the cloud simulator CloudSim with some edge computing specific extensions, such as iFogSim [14], and the extensions of iFogSim. The other two types of simulators are FogNetSim++[15], which is developed based on discrete event simulator OMNeT++, and the self-developed Java tool (consisting of FogTorch and FogTorchII [16]). Compared to simulation platforms, emulation systems can support both repeatable and controllable experiments with real applications. Currently, emulators for ECCI are relatively few, such as Emufog [17], Fogbed [18], and MockFog [19]. However, these emulators focus more on computing and network emulation, and they do not provide a one-stop service for orchestrating, deploying, and monitoring applications.

[1][Online]. Available: https://v1-4.docs.kubeedge.io/en/
[2][Online]. Available: https://www.lfedge.org/projects/baetyl/

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

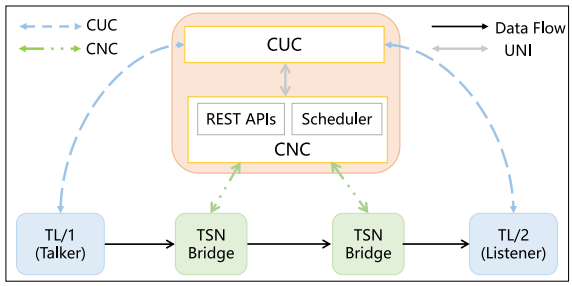


Fig. 1. Sample topology of the TSN components in Cisco IE 4000 Switch.

B. TSN Platforms

As a promising network technology, TSN [20] allows the coexistence of time-critical and best-effort traffic on the same network and provides guaranteed data transport with bounded low latency for time-critical traffic. That is, TSN can seamlessly merge the operation and information technology in a smart factory, and the hard real-time demand of operation technology can be guaranteed in the network layer [21]. Currently, TSN has demonstrated practical value in many mission-critical application scenarios. The fully centralized model of TSN-supported Cisco IE 4000 switch[3] is shown in Fig. 1. It defines a user network interface (UNI), which enables users to specify data flow requirements without knowledge of the network, thereby making the network configuration transparent to users. TL/1 and TL/2 are respectively the talker and listener of the data flow. TSN bridge is the TSN-supported switch. Centralized user configuration (CUC) and centralized network configuration (CNC) are installed in the same LAN as the TSN bridge. The workflow of this configuration model is as follows. First, CUC interacts with CNC through the REST APIs to provide the communication requirements of the data flow proposed by the user. CNC then summarizes all the requirements and calculates the optimal path for each data flow. Finally, the calculated plan is transmitted to each TSN bridge. Based on the configuration information received from CNC, the TSN bridge can use an appropriate strategy when forwarding flow frames to guarantee the latency of time-critical flow. Furthermore, the popular simulators are TSN-specific extensions to the popular OMNeT++ framework for network simulations [22], including TSimNet [23] (i.e., focusing more on non-time-based components of the TSN mechanisms), OPNET [24], NeSTiNG [25] (i.e., focusing more on implementing the time-triggered scheduling mechanism of IEEE Std 802.1Qbv). However, real applications cannot run in these simulations, and thus the characteristics of real applications deploying in TSN cannot be obtained.

In summary, the state-of-the-art platforms lack considering the high real-time latency demand of industrial operations, which severely hinders the development of industrial operations and may even cause industrial accidents. To make improvement to this problem, we developed the IndustEdge platform to significantly reduce the system level latency of industrial applications, whose comparison with similar platforms in the market is summarized in Table I.

[3][Online]. Available: https://www.cisco.com/c/en/us/td/docs/switches/lan/cisco_ie4000/tsn/b_tsn_ios_support/b_tsn_ios_support_chapter_01.html

III. IndustEdge Platforms

A. Overall Architecture

In designing the ECCI platform IndustEdge, we consider the following three layered architecture, as shown in Fig. 2.

1) The *infrastructure layer* is the cornerstone of IndustEdge, including end, edge, and cloud three-layer devices. End devices are either embedded in or located close to the field devices, such as machine tools, sensors, and cameras. Based on sensory inputs from the end devices and edge devices can compute and store data to dynamically adjust the execution strategy of the industrial equipment. Cloud devices, either public or private clouds, have nearly infinite resources compared to the edge devices, but locate far away from the end devices. Considering the high real-time latency demand of industrial operations, the TSN protocol is applied to guarantee the network level latency.
2) The *platform layer* is the core of IndustEdge, which supports the industrial intelligent applications in the application layer upward, and docks the industrial edge cloud collaborative devices in the infrastructure layer downward. It provides ECCI algorithms library, devices management, and agile development and rapid deployment of ECCI applications. See Sections III-B, III-C, and III-D for detailed description about its design and implementation.
3) The *application layer* is the interface of IndustEdge, which supports typical ECCI applications for smart industry, such as smart grid, smart manufacturing, smart water systems and so on. IndustEdge aims to support applications constructed of multiple microservices and connected in the form of a directed acyclic graph (DAG).

Next, we will introduce the detailed design of IndustEdge platforms based on the overall architecture above.

B. Prototype Platform

As shown in Fig. 3, the prototype platform consists of the *platform-level* and *infrastructure-level* components, running in the platform layer and infrastructure layer, respectively:

1) Platform-Level Components: Platform-level components manage all entities in the system, including users, infrastructure, applications, and others; make global decisions (e.g., orchestration); detect and respond to events (e.g., monitoring the running status of each component in real-time, and making relevant actions for some events). They can be deployed on any device that can be accessed by all other devices in the infrastructure layer. These components are *API server*, *database*, *dashboard*, *orchestrator*, *manager*, *platform hub*, and *algorithms library*, as described below.

1) The *API server* is the data center of the platform that serves the REST operations for validating and configuring data for IndustEdge objects. The physical topology creation and data

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

TABLE I
INDUTEDGE VERSUS SIMILAR PLATFORMS IN THE MARKET

	ECCI Oriented			IoT Oriented			**ECCI Industry Oriented**
	KubeEdge	OpenYurt	Baetyl	Link IoT Edge	Azure IoT Edge	EdgeXFoundry	**IndustEdge**
TSN	×	×	×	×	×	×	√
SDK	×	√	√	√	√	√	√
Drag-and-drop development	×	×	×	√	×	×	√
Cost of large-scale deployment	high	high	high	high	high	high	**low**
System architecture-supported	X86	X86	X86/ARM	X86/ARM	X86/ARM	X86/ARM	**X86/ARM**
Intelligent algorithms and models	weak	weak	weak	medium	medium	weak	**strong**
CloudOnly/EdgeOnly/ECCI mode	√	√	√	√	√	√	√
Edge edge collaboration mode	×	×	×	×	×	×	√
Real-time performance	weak	weak	weak	medium	medium	medium	**strong**
Data privacy protection	medium	medium	medium	strong	strong	medium	**strong**
Corresponding emulation platform	×	×	×	×	×	×	√

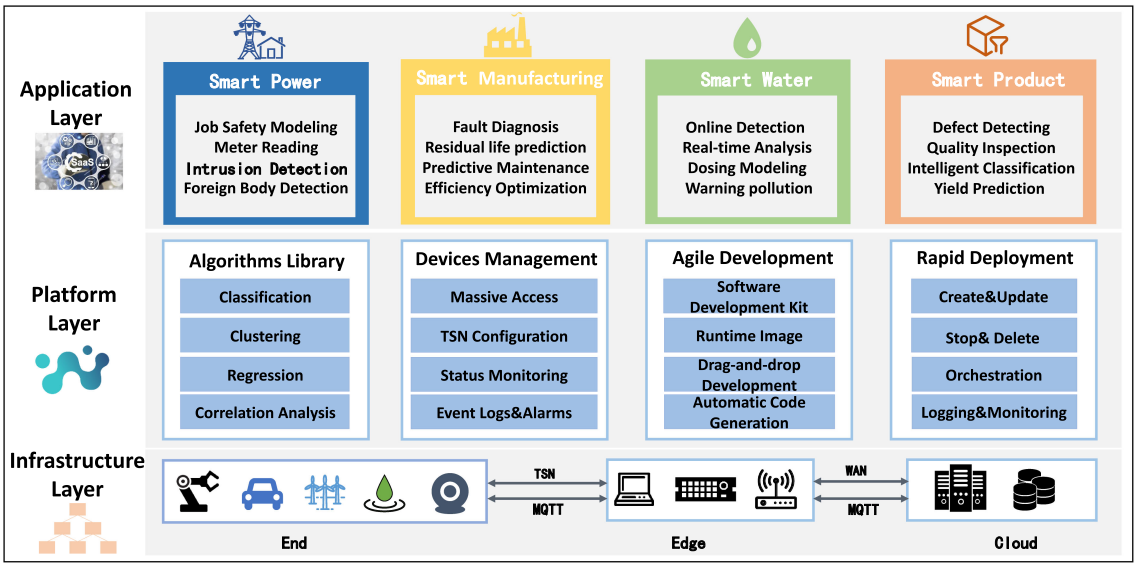


Fig. 2. Overall architecture of IndustEdge.

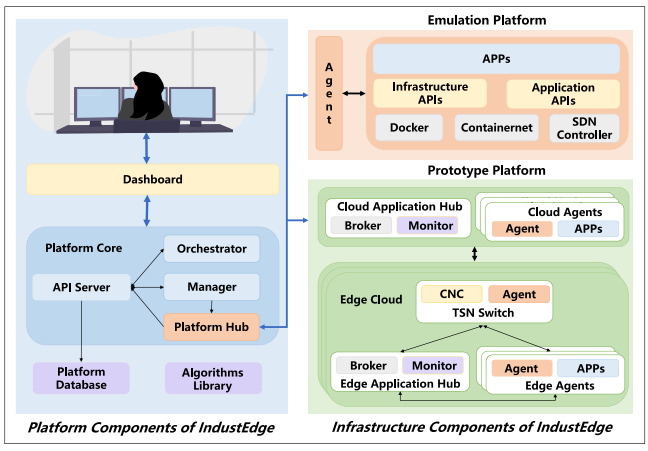


Fig. 3. Components of IndustEdge for prototype and emulation.

flow design of TSN's CUC are also included in the infrastructure and application objects, respectively. Besides, it is the only component that directly operates on the *database*, which uses a consistent and highly-available relational database to store all platform data.

2) The *dashboard*, based on the *API server*, is provided to users to facilitate operation and management, such as developing a drag-and-drop way to create an application and the real-time status display of applications and infrastructure.

3) The *orchestrator* obtains the deployment scheme of the application (i.e., the device that each microservice of the application is placed on) and configuration information of the TSN. We provide a unified SDK for orchestration algorithms so that users can customize their own industrial scenarios to reduce the system level latency [26].

4) The *manager* is responsible for the complete life cycle management of the applications (e.g., deployment, update, stop) by interacting with *infrastructure-level components* through the *platform hub*, which is the interaction center for different information.

5) The *algorithms library* hosts images of different ECCI algorithms in the image repository `Harbor`, which includes

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

the common algorithms for different analytic tasks, such as classification, clustering, correlation analysis.

2) Infrastructure-Level Components: The infrastructure-level components support the deployment of various ECCI applications, maintain application life cycle management, and dynamically adjust the TSN configuration by interacting with the *platform-level components*. Users can register a central cloud and multiple edge clouds. A central cloud includes several physical devices or VMs, while an edge cloud includes several end/edge devices and a TSN switch. The *infrastructure-level components* are *cloud/edge agents*, the *cloud/edge application hub*, and the *TSN switch*, as described below.

1) In the *cloud/edge agents,* the platform generates an *agent* for each registered device. After one-click installation of the *agent*, each device communicate with *platform-level components* through the *platform hub*. Moreover, each device can be used to deploy several applications or microservices according to the placement strategy generated by the *orchestrator*.

2) In the *cloud/edge application hub,* a *broker service* provides global control and data communication for applications within the central cloud and edge cloud. It realizes different collaboration modes, such as edge-cloud collaboration and edge-edge collaboration. A *monitor service* is included for uploading status information of the *broker service* to the *platform-level components*. Besides, the *edge hub* is bridged to the *cloud hub* so that microservices on the edge cloud only need to be concerned with the connection to the local *edge hub*.

3) In the *TSN switch,* the *agent* here is completely the same as that of the *cloud/edge agents*. The *manager* transmits the information of physical topology and data flows generated by *orchestrator* to the *agent*, and then the *scheduler* is called by the *agent* through the REST APIs of the CNC to generate the specific network configuration of the TSN switch.

Users are extremely maneuverable with their registered infrastructure, and the architecture has dual offline support. Even if the link is in poor condition or the *platform-level components* itself malfunctions, the deployed applications will not be affected. Bridging between the *cloud* and *edge application hub* enables data synchronization when the network is restored.

C. Emulation Platform

Due to the high cost of large-scale deployment, an emulation platform of IndustEdge is proposed to provide system design guidelines for possible large-scale deployment.

The essential difference between the emulation and prototype platform is that the infrastructure of emualtion platform is simulated and the infrastructure of prototype platform is real, as shown in Fig. 3. Therefore, the goal of the emulation platform is to emulate infrastructure (including devices and network) and run the real-world ECCI applications in the emulated infrastructure like a prototype platform.

To emulate the infrastructure, we adopt `Containernet`,[4] which is a branch of the famous Mininet network emulator and focuses on the field of edge computing. `Containernet` supports `Docker Containers` as virtual devices to emulate the central cloud and multiple edge clouds with different resources (e.g., CPU and memory). It uses a `SDN Controller` to create the link (e.g., bandwidth and delay) between the device and switch. To facilitate the construction of the infrastructure and the operation of applications, we encapsulate the infrastructure and application APIs based on `Containernet`. Infrastructure APIs include the topology, cloudInstance, edgecloudInstance, tsnSwitch, resource, and link. Application APIs include the image and resource. The *platform-level components* send the user configuration of the infrastructure and applications to the *agent* through the *platform hub*. The *agent* invokes APIs to create the infrastructure and run the applications. Meanwhile, the *agent* monitors the infrastructure and application status in real-time and sends them to the *platform-level components* for real-time display to users.

D. Implementation Details

This section focuses on the implementation details of the communication, life cycle management of applications and infrastructure along with resource allocation of the emulation platform. The workflow of IndustEdge is also demonstrated.

1) Communication: To enable edge-cloud communication of application and *platform-infrastructure components* interaction of InudstEdge, *application hub* and *platform hub* are developed based on `EMQ X`,[5] an open source MQTT broker for IoT. It has the characteristics of reliable message transmission, low bandwidth, and low energy consumption as well as the support for large-scale connections, authentication method, and offline transmission. Furthermore, We support the software development kit (SDK) and automatic routing, which enables users to focus on the microservices processing logic without knowledge of the placement strategy of the microservices, the IP and port of each device, and the topic of communication between the microservices, thereby making communication transparent to users. IndustEdge realizes the lines of application code $2\times$ less than that of initial development effort.

2) Life Cycle Management: To facilitate the management of applications and infrastructure, we provide `Docker Compose` and `K8s+K3s` to run and maintain applications in the infrastructure. The `Docker Compose` solution is convenient and suitable for scenarios with one edge device in the edge cloud. The `K8s+K3s` solution has powerful intelligent operation and maintenance functions, which is suitable for scenarios with multiple devices in the edge cloud. `K8s` and lightweight `K3s` are deployed in the central cloud and edge cloud, respectively. In addition to basic functions, such as starting, stopping applications, the two schemes also provide fault-tolerant functions, like restart after downtime and zero shutdowns when updating applications. Moreover, there are two ways to directly upload `YAML` (a human-readable data-serialization language) file or automatically generate `YML` file by drag-drop to create the

[4][Online]. Available: https://github.com/containernet/containernet

[5][Online]. Available: https://www.emqx.io/

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

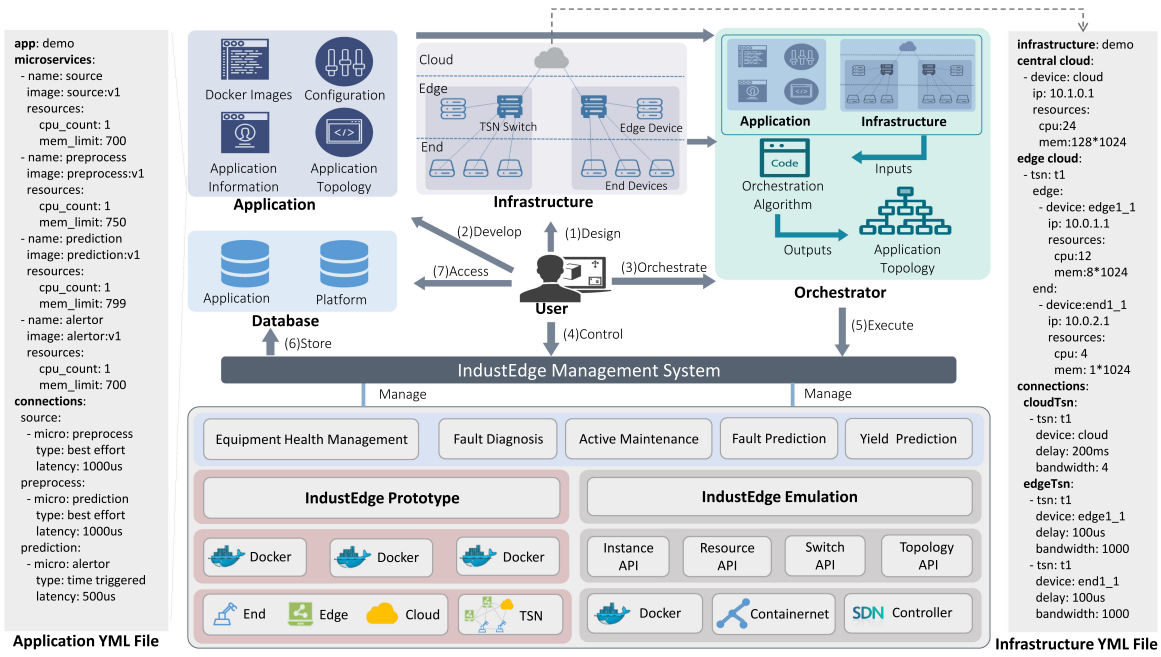


Fig. 4. Workflow in the development and management of ECCI applications on IndustEdge.

applications and infrastructure. The YAML files are shown in Fig. 4.

3) Resource Allocation: For adaptively resource allocation on different emulated devices, the IndustEdge emulation platform allocates the actual resources of each emulated device according to $a/b * c$. Here, a, b, and c represent the resources required by the device, the resources required by all the devices, and the resources available at the location of the emulation platform, respectively.

4) User Interface: As shown in Fig. 4, we integrated the prototype and emulation platform. The following steps explain how IndustEdge can be used in detail. 1) In the *dashboard*, users first design the infrastructure and applications as required, set up the category (prototype or emulation) for the infrastructure, and install the *agent* according to the category. 2) Then, users click the orchestration operation and the platform will automatically input the application and infrastructure to the *orchestrator*, which generates the placement scheme and TSN configuration. 3) After orchestration is complete, users click the deployment operation. If it is for the prototype platform, then the *manager* sends the deployment plan and TSN configuration plan to the *agent* and TSN switches through the *platform hub*; otherwise, the *manager* merges the placement policy and TSN configuration information into one deployment plan, and then sends the plan to the *agent* of the emulation platform for infrastructure construction and application startup. 4) Finally, the application processing results and platform information are stored in the *database* for big data analysis and visualization.

IV. System Level Latency Analysis

Fig. 5 shows the breakdown architecture of the system level latency. Control information starts from the source host, crosses

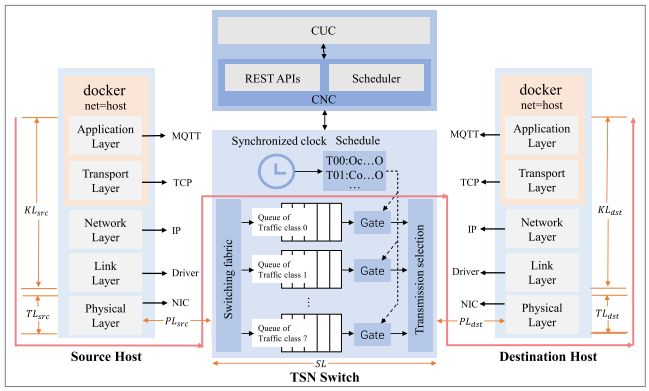


Fig. 5. Breakdown architecture of the system level latency.

the switch, and finally arrives at their destination host. The total latency L includes the following four components.

1) *The protocol processing latency* (KL), which is the processing latency associated with the protocol stack and latency caused by the Docker container interacting with the host of source and destination host.
2) *The transmission latency* (TL) on the source and destination host.
3) *The propagation latency* (PL) on transmission medium.
4) *The switch latency* (SL) caused by the packet forwarding in the switch [27].

Since the industrial operation application studied in this article only sends a control instruction with the content of "0/1," the latency of a packet equals the latency of the application. The system level latency can be presented by the following formula:

$$L = KL_{src} + TL_{src} + PL_{src} + SL$$

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

$$+PL_{dst} + TL_{dst} + KL_{dst}.$$

A. Fine-Grained Decomposition of the System Level Latency

KL_{src} starts from the moment when the application on the source host sends out a message to the socket interface. Then the message is processed by the protocol stack and finally the network interface card (NIC) driver fetches the packet from the host memory and sends it to the physical layer. Since the application is running in the docker container using the network configuration as the host in this article, KL_{src} also contain the latency incurred during the interaction between the docker and host at the application and transport layer in the source and destination host.

TL_{src} is the latency it takes to push all bits of the packet from the source host to the outgoing link. TL_{src} can be calculated by the following equation: $TL_{src} = s/r + w$, where s is the packet size, r is the transmission rate and w is the waiting time.

PL_{src} refers to the latency of propagating packets in the communication media (copper, optical fiber, or wireless channel) on each link. It is proportional to the distance between the source and the destination host, and can be presented by the following formula for each link segment: $PL_{src} = l/p + w$, where l is the distance, p is the propagation speed and w is the waiting time. Propagation speed varies in different communication media.

SL is the latency caused by packet forwarding in the TSN switch. As depicted, a switching fabric identifies the frames on the ingress. The frames will be specifically assigned to one of the queues associated with the same port. To realize time-triggered communication, TSN defines a specific shaping mechanism on how frames are selected for transmission from a queue [20].

The receiving process on the destination host is the reverse process of packet sending. PL_{dst} refers to the latency of propagating packets in the communication media from the link to the destination host. TL_{dst} is the amount of time required to receive all bits of a packet from the link to the destination host. KL_{dst} is the time gap between the moment when the NIC driver starts to copy the packet into memory and the moment when the application at the destination host receives this packet.

B. System Level Latency Analysis Based on Profiling

According to the description above, we know that SL can be ensured by the specific shaping mechanism of TSN. The literature [28] shows that when Docker runs on the host based on Linux container using the network configuration as the host, the overhead is minimal and the performance is basically the same as the host. However, KL, TL, and PL all have waiting time, which would be impacted by the available CPU, memory, IO, NIC, operating system (OS), protocols, and other factors. In practice, it is difficult to take all factors into full consideration and impossible to formalize the latency. Therefore, we conduct extensive simulations to profile the relationship between the system level latency and different stress configurations of CPU, memory, IO, disk, and MQTT transmission of the device. Based on the profiling results, we empirically analyze the system level latency as follows.

TABLE II
CONFIGURATIONS OF MINISERVER AND RASPBERRY PI

		Miniserver	Raspberry Pi
Motherboard		H110D4-P1	3 Model B+
NIC	Model	RTL8111/8168/8411	Gigabit Ethernet
	Bandwidth	1000Mb/s	300Mb/s
CPU	Model	Intel(R) Core(TM) i7-7700	ARM Cortex-A53
	Freq	4.2GHz	0.9GHz
	Cores	8 (already use 1 core)	4
RAM	Model	Micron DDR4	LPDDR2 SDRAM
	Memory	8G (already use 0.89G)	1G
Storage		120G (already use 97.01G)	30G
OS	Version	Ubuntu 18.04 bionic	Raspbian 10 buster
	Kernel	x86-64 Linux 4.15.0	armv7l Linux 4.19.66

1) Setup: We set up a system based on IndustEdge consisting of one edge device (i.e., a Miniserver), one TSN switch (i.e., a Cisco IE4000 TSN-supported switch) and one terminal device (i.e., a Raspberry Pi). The control application is simulated as follows: the Docker container (Docker 19.03.5) in Miniserver sends a control information "0" to the Docker container (Docker 19.03.2) in Raspberry Pi using the MQTT protocol (Mosquito 1.4.15). The control flow is set as TSN time-triggered traffic (i.e., 500 us) to complete data forwarding by the TSN switch. Table II shows Motherboard, NIC, CPU, RAM, storage and OS configurations of Miniserver and Raspberry Pi.

2) Parameter Settings: The Raspberry Pi is only used to receive control instructions. We use the Stress tool on the Miniserver to various workloads on CPU, memory, IO, and disk to capture system level latency of the control application under different stress of CPU, memory, IO and disk. We also add other Docker containers on the Miniserver to capture system level latency of the control application under different MQTT transfers. And the latency requirement of the control application is set as 15 ms in our simulations.

3) Results: CPU-memory: First, we start the Docker container of the control application on Miniserver only. Then, the Stress tool is used to load memory and CPU to obtain the latency of the control application with CPU stress ranging from 1 to 8 cores under memory stress of 1.86, 3.56, 4.62, 6.19, and 7.22 G, respectively. Fig. 6(a) shows that the latency is affected by CPU utilization. When the CPU stress is higher than 7 cores (i.e., CPU usage is higher than 98%), the latency requirement (i.e., 15 ms) is no longer met. The memory utilization has little effect on the latency. When the CPU utilization is low, the latency does not increase with the increase of memory loads, and always meets the latency requirements.

MQTT-CPU: First, we start the Docker container of the control application and two Docker containers of the MQTT stress application, one for sending data in a loop and one for receiving data on the Miniserver. Then, the stress tool is used to load memory and CPU. We fix memory load as 5.37 G and obtain the latency of the control application with MQTT stress of 0, 465 B, 112 K, 368 K, 2.04 M, 3.84 M, 4.59 M, 6.61 M under CPU stress of 2, 4, 5, 6 cores, respectively. Fig. 6(b) shows that the latency is affected by both MQTT transfers and CPU. When CPU stress is low (i.e., 2 and 4 cores), the latency requirement

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

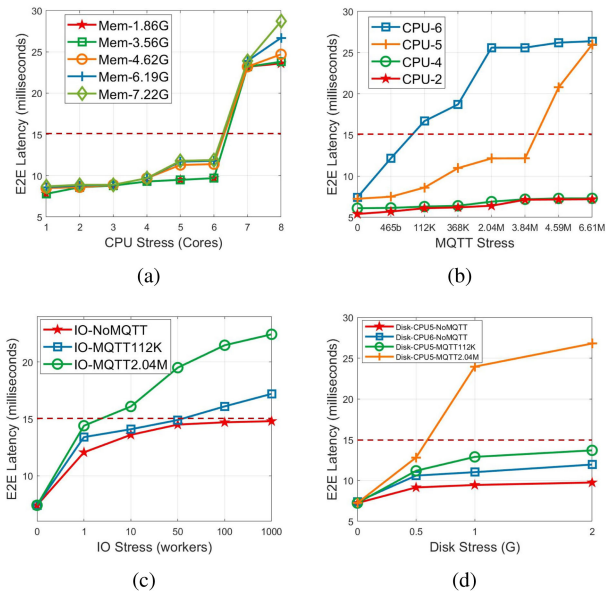


Fig. 6. System level latency of the control application under different resource stress.

can be guaranteed at all MQTT transfers. However, when the CPU stress is large (i.e., 5 and 6 cores), the latency is no longer guaranteed when the MQTT transfers exceed a certain value (*i.e.*, 4.59 M under 5 cores and 112 K under 6 cores). It is worth noting that all cases where the latency requirement are not met, have a CPU usage of at least 98%.

IO-MQTT: First, we start the Docker container of the control application and two Docker containers of the MQTT stress application on the Miniserver. Then, the Stress tool is used to load memory, CPU and IO (spawn N workers, repeatedly calls sync() to write the contents of memory to hard disk). We fix the memory as 5.37 G and CPU as 6 cores, and obtain the latency of the control application with IO stress of 0, 1, 10, 50, 100, 1000 workers under MQTT transfers of 0, 112 K, 2.04 M, respectively. Fig. 6(c) shows that latency is strongly correlated with MQTT and weakly correlated with IO. Latency is guaranteed under IO stress alone. However, under the dual stress of IO and MQTT, latency is no longer guaranteed under CPU stress as 6 cores. We still find that all cases where the latency requirement are not met, have a CPU usage of at least 98%.

Disk-MQTT-CPU: First, we start the Docker container of the control application and two Docker containers of the MQTT stress application on Miniserver. Then, the Stress tool is used to load memory, CPU, IO and disk (spawn four workers, each writing a fixed size temporary file to the current directory). We fix the memory as 5.37 G and obtain the latency of the control application with disk stress of 0, 0.5 , 1 , 2 G file under MQTT transfers of 112 K, 2.04 M, and CPU of 5, 6 cores. Fig. 6(d) shows that latency is strongly correlated with MQTT and weakly correlated with disk. Latency is guaranteed under disk stress alone. However, under the dual stress of disk and MQTT, latency is no longer guaranteed when MQTT transfers exceed 2.04 M under CPU stress as 5 cores. Similarly, we also find that all cases where the latency requirement are not met, have a CPU usage of at least 98%.

4) Summary: Fig. 6 shows that the latency of the control application is strongly correlated with CPU and MQTT loads, but weakly correlated with IO and disk loads. The latency requirement (i.e., 15 ms) cannot be guaranteed when the CPU utilization of the Miniserver is higher than 98%. Considering the heterogeneity of the edge devices, it is difficult to test the latency of application layer on all edge devices for the control application. Therefore, it is impossible to give the latency upper bound and latency guarantee conditions applicable to all edge devices. For achieving latency reduction of the control application as much as possible, we set up a heuristic algorithm in the orchestration component of IndustEdge. The main idea is that the container that sends control instructions selects the device with CPU utilization less than 50% and MQTT transfers less than 2 M/s.

V. Case Studies

To evaluate the effectiveness of IndustEdge, two case studies have been carried out. The first case study (*video analysis for real-time intrusion detection*) shows that IndustEdge can not only accurately detect intrusions, but also respond quickly to intrusion behavior through the demo system built in the laboratory. The second case study (*time series analysis for real-time predictive maintenance of cutting tools*) validates that IndustEdge can effectively detect and respond faster to anomalies, through deploying it in the manufacturing workshops of an aviation engine manufactory.

A. ECCI Video Analysis for Real-Time Intrusion Detection

1) Background: Intrusion detection is crucial for the safety of industrial operations and properties in manufactories. The prompt alarms for intrusions can prevent the impact of equipment theft or severe accidents. Traditional manual inspection and remote monitoring systems are time-consuming and expensive, and they are difficult to meet the accuracy and real-time requirements. In recent years, with the rapid development of machine vision technology, smart video monitoring technology has gradually become the mainstream approach. The captured video are delivered to the cloud with abundant computing resources, where the contents are then processed to obtain the intrusion behavior. Finally, the control information is transmitted to the alertor for alarm.

However, when a camera with 1280 $\times$ 1024 pixels records at 20 frames per second, it generates 50 G of data per day, which poses great pressure on data delivery [5]. Moreover, the video frames and the control information are transmitted in the same network and the video frames occupy most of the channels; thus, it is difficult to ensure the hard real-time requirement of the control information. Instead, such a video monitoring system can be better deployed by our ECCI platform IndustEdge to achieve ECCI video analysis with lower response latency, more agile development and more rapid deployment.

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

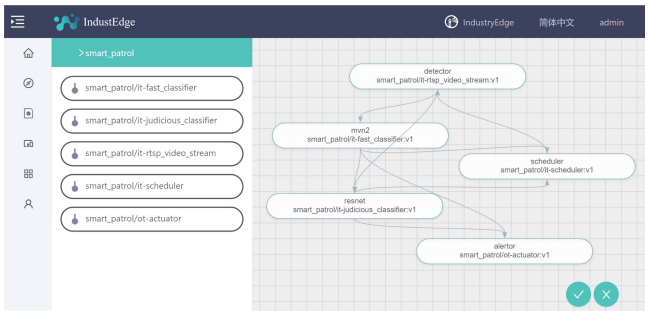


Fig. 7. User interface for the configuration of ECCI applications.

2) Edge Cloud Collaborative Application: The workflow of the intrusion detection application in IndustEdge is shown in Fig. 7, where the detection algorithm adopted the ECC scheme proposed in a prior study [29]. After the *detector* reads the video stream for the inter-frame difference, the image is sent to *mvn2* or *resnet* according to the *scheduler*'s dispatch rule (originally *mnv2*). *Mnv2* is the low-accuracy classifier and subscribes the image packages from the *detector* to inference. It decides whether to publish the image to *resnet* for further inference based on the confidence of the result. *Resnet* is the high-accuracy classifier and subscribes the image packages from the *detector* or *mnv2* to inference. The *scheduler* subscribes the queue length information sent by *mnv2* and *resnet* and determines where the *detector*'s image is sent. Finally, once the recognition results of *mnv2* and *resnet* are intrusion behaviors, control information is sent to *alertor* for alarming. The control flow was set as TSN time-triggered traffic (i.e., 500 us) to complete data forwarding by the TSN switch. The other flows were also set as TSN time-triggered (i.e., 1000 us) to complete data forwarding by the TSN switch.

3) Integration of Hardware and Software: As shown in Fig. 8(a), we constructed a multilayer and heterogeneous IndustEdge prototype platform. It is composed of one cloud server (a virtual server on the public cloud) and multiple heterogeneous edge devices (one x86 based PC, i.e., a MiniServer, and an ARM-based development board, i.e., an Nvidia Jetson AGX Xavier) and end devices (one ARM-based development boards, i.e., one Raspberry Pi as alertor, and one camera), and the TSN-supported switch (one Cisco IE 4000 switch), where the camera is directly connected to the Xavier via USB. Detailed device specifications and bandwidth settings are summarized in Table III. The integration of the IndustEdge prototype platform with hardware is divided into the following four steps: First, any authenticated user can launch the installation package of the *platform components* in the central cloud with one click to complete the installation; Then, the user register all devices in turn, after which, the platform will generate *infrastructure components* installation package to devices. And, the user downloads the installation package to each device and install it. Next, the user creates the application and configures TSN switch. Finally, the user deploys the applications with one click to get the predictive results in real time.

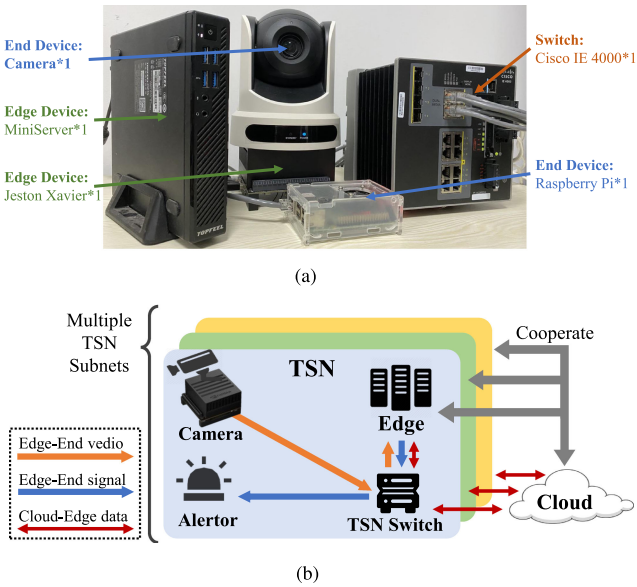


Fig. 8. Prototype and emulation platforms of IndustEdge. (a) Prototype platform. (b) Emulation platform.

TABLE III
DETAILS OF THE PROTOTYPE PLATFORM

(a) Device specifications

	Device	CPU Model	Freq (GHz)	Cores (count)	Mem (G)	Storage (G)
Cloud	Public Cloud	IX*	2.5	24	128	2150
TSN Switch	Cisco IE 4000	\	\	\	\	\
Edge	Miniserver	IC*	4.2	8	8	120
	Jeston Xavier	V8.2*	2.2	8	32	32
End	Raspberry Pi	A53*	0.9	4	1	30
	Camera	\	\	\	\	\

* **IX**:Intel (R) Xeon (R) Platinum 8163; **IC**:Intel (R) Core (TM) i7-7700; **V8.2**:ARM V8.2; **A53**:ARM Cortex-A53.

(b) Communication bandwidth (Mb/s)

	Respberry Pi	Xavier	MiniServer	Public Cloud
Respberry Pi	N/A	331	93.2	4.36
Xavier	94.7	N/A	93.4	3.80
MiniServer	93.4	93.8	N/A	3.79
Public Cloud	3.36	3.18	3.36	N/A

As shown in Fig. 8(b), the *platform* and *infrastructure components* of emulation software are all deployed in the public cloud (e.g., 2.5 GHz, 24 cores, 128 G RAM, 2150 G storage). After successful installation, it can automatically detect the available CPU, memory, and storage on the device. The user interacts via the user-friendly UI, inputs the number of TSN subnets for emulation, and then sets the number and resource configuration of devices for each subnet in turn (a subnet only have one TSN switch). After all the settings of each subnet, these emulated devices and their configured resources will be generated in the form of Docker container. It is worth pointing out that the emulation platform can emulate any bandwidth and delay from the device to the switch. However, if the sum of CPU, memory and storage requirements of all devices is higher than the available resources of the public cloud, the emulation platform will call

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

the *Resource Allocation Algorithm* in Section III-D to calculate the actual resources acquired by each device.

We conducted experiments on both the prototype and emulation platforms.

Prototype platform-based experiments

1) We obtained the latency of the video and control flows in the ECC (i.e., the *detector*, *mnv2*, *resnet*, and *alertor* were placed at Xavier, MiniServer, the cloud, and Raspberry Pi, respectively), CloudOnly (i.e., the *detector* microservice was the same as the ECC, but all the image packages were transmitted to the cloud and classified by the *Resnet*), and EdgeOnly (i.e., the *detector* microservice was the same as ECC, but all the image packages were classified by the local edge classifier based on the *Mnv2*) settings under whether the switch supports the TSN protocol. Furthermore, to validate that TSN switch of IndustEdge can guarantee the low latency of the control flows in the network layer, the network loads were created by starting 1000 processes on an additional edge device (i.e., a MiniServer). The 1000 processes sent the same size files to the Alertor simultaneously using the TCP service. To create different network loads, the file size was set as 100, 200, 400, 600, and 800 M, respectively. This data flow was also set as time-triggered (i.e., 5000 us) to complete data forwarding by the TSN switch.

Emulation platform-based experiments

1) Network bandwidth selection: We emulated one central cloud, two edge devices (i.e., the Xavier and MiniServer), two end devices (i.e., the Camera and Alertor) and one switch. In the EdgeOnly setting, we fixed the delay between edge devices and TSN switch as 1 ms (Edge-Bd), and changed the bandwidth between the edge and TSN switch from 100 Mbps to 1000 Mbps. We obtained the latency of the video and control applications under both TSN-supported and TSN-unsupported scenes.
2) Computing resource selection: We emulated one central cloud, two edge devices (i.e., the Xavier and MiniServer), two end devices (i.e., the Camera and Alertor), and one switch. In the ECC and TSN-supported setting, we assigned 1 CPU cores for each end device and changed the number of CPU cores for each edge device from 0.5 to 10. Then, we fixed the number of CPU cores for each edge device as 8 (TSN-ECC-Edge-CPU) and changed the CPU cores of end devices from 0.5 to 10 (TSN-ECC-End-CPU).
3) Number of TSN subnets (Multi-TSN): We fixed the delay between the central cloud and TSN switch as 100 ms, the bandwidth as 10 Mbps, the delay between the edge and TSN switch as 1 ms, and the bandwidth as 100 Mbps. Each subnet contained one switch, two edge devices, and two end devices. We changed the number of TSN subnets from 1 to 10 to obtain the latency of the video and control applications under CloudOnly and ECC.

4) Evaluation Metrics: We collected real surveillance video streams from YouTube live for the performance evaluation. Specifically, more than 170 hours of video streams from 14 cameras with 1080 p resolution and 30 fps frame rate were collected. We selected 130 hours of video data for the training of specific CNNs (with MobileNet-v2 and ResNet152 for classification). The remaining data were used for evaluation. To demonstrate the effectiveness and efficiency of IndustEdge, we used the predictive accuracy (i.e., the ResNet-152 was treated as the ground-truth, and we used the F-score to measure the accuracy of intrusion detection [29]), and the average latency of video and control flows.

TABLE IV
COMPARISON OF COLLABORATION SCHEMES ON INDUSTEDGE

Type	TSN?	Video Accuracy	Video Latency	Bandwidth cost
CloudOnly	Yes	100%	54.164s	423.6MB
CloudOnly	No	100%	54.164s	423.6MB
EdgeOnly	Yes	69.32%	0.101s	0MB
EdgeOnly	No	69.32%	0.102s	0MB
ECC	Yes	89.21%	1.585s	93.2MB
ECC	No	89.21%	1.994s	93.2MB

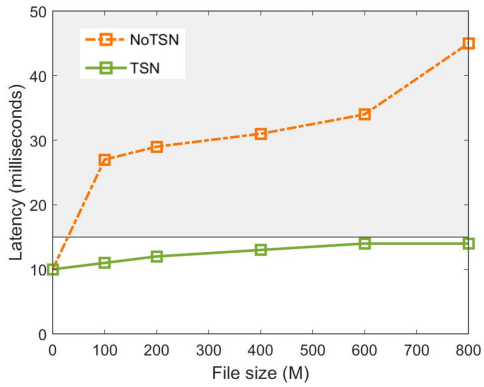


Fig. 9. Latency of control application under different network loads.

5) Experiment Results: Table IV shows the results of video application in the prototype platform. Compared with the traditional CloudOnly intrusion detection, ECC achieves 27.08× faster detection response, 4.55× fewer bandwidth cost, and maintains an acceptable detection accuracy. Compared with the EdgeOnly intrusion detection, ECC improves the detection accuracy by 19.89%. That is, ECC makes a good tradeoff between the accuracy and the detection latency.

Fig. 9 shows the latency results of the control application in the prototype platform. In the case of no network loads (i.e., file size=0 M), the latency demand of the control application can be satisfied regardless of TSN support. As we can see, with the increase of the network load, the control latency in the TSN supported prototype platform increases slowly and keeps lower than 15 ms even when there is a heavy network load (i.e., file size=800 M), which always satisfies the latency requirement. Without TSN, the control application latency increases dramatically. When the file size is about 30 M, the latency surpasses 15 ms and no longer meets the latency requirement.

Fig. 10 shows the results of average latency of the video and control flows in the emulation platform.

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

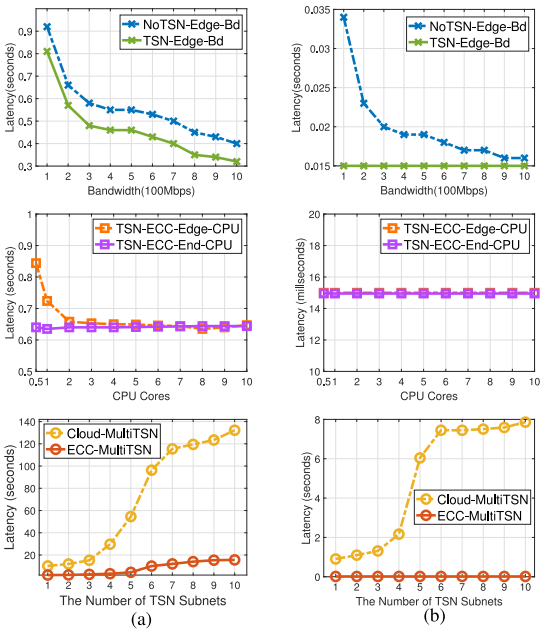


Fig. 10. Average latency of video and control applications under different parameters. (a) Latency of video flows. (b) Latency of control flows.

1) Network bandwidth selection: In the TSN-unsupported scene, setting the bandwidth as 1000 Mbps is the best choice. At this time, the low latency of control application can be guaranteed and the latency of video application is the lowest. In the TSN-supported scene, setting the bandwidth as 300 Mbps is the best choice based on the tradeoff of application latency and the bandwidth consumption (i.e., the greater the bandwidth, the higher the cost). At this point, the latency of video application drops to the first inflection point within the acceptable range. And the latency demand of control application can still be guaranteed. Moreover, these results further demonstrate the effectiveness of TSN.
2) Computing resource selection: In terms of the selection of CPU cores for edge and end devices, the edge device with 8 cores and the end device with 1 core are the best choice, which achieve the minimum latency of video application.
3) Multiple TSN subnets: In the ECC setting, control latency is always guaranteed, but video latency starts to spike when there are 6 TSN subnets. So, up to 5 TSN subnets share a central cloud under this configuration. In the CloudOnly setting, control latency and video latency both experience a spike when there are 4 TSN subnet. So, up to 3 TSN subnets share a central cloud under this configuration. Moreover, these results further demonstrate that ECC is superior to CloudOnly.

B. ECCI Time Series Analysis for Real-Time Predictive Maintenance of Cutting Tools

1) Background: The workpieces processed by the machine (e.g., the cutter) of the aviation engine manufactory are mostly

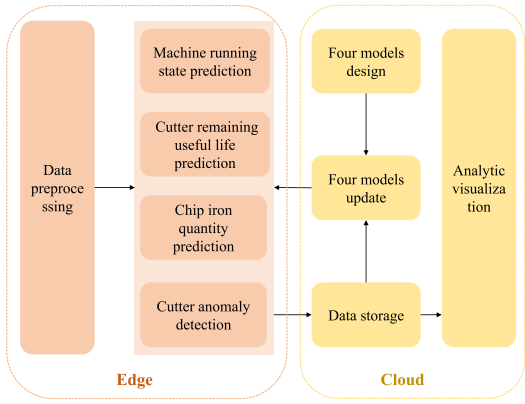


Fig. 11. Four applications of real-time predictive maintenance of cutting tools.

expensive parts in aero engines. The quality and scrap rate of the cutting process are directly impacted by the health condition of the cutter, which, however is difficult to monitor with human eyes. Currently, the detection of damaged cutters is mostly done by experienced workers, which still has high scrap or low cutter efficiency. To improve the detection efficiency while reducing the manual labor, a predictive maintenance system with low latency is highly demanded for the cutters. To this end, based on IndustEdge, we have developed such a predictive maintenance system based on the signals of the vibration sensor and current sensor, which has been deployed in the manufacturing workshops of an aviation engine manufactory. It can not only accurately predict the health condition of the cutters, but also control the machine within milliseconds in the case of cutter damage, thus, greatly improving the production efficiency.

2) Edge Cloud Collaborative Application: As shown in Fig. 11, we developed four edge cloud collaborative applications: machine running state prediction based on support vector machine, cutter remaining useful life prediction and chip iron quantity prediction based on random forest, and cutter anomaly detection based on K-means. Edge devices performed data preprocessing (i.e., transformed the original high-frequency signal data to the time/frequency domain data, and extracted the features of the corresponding statistics to get the feature vector). And edge devices also ran the application model (i.e., obtained the predicted values of the four applications, immediately controlled the machine tool once detected anomalies of the cutter, and transmitted the feature vectors and results to the central cloud). The central cloud performed dynamic model updating (i.e., constantly updated the model according to new data and broadcasted the new model to the edge), data storage, result analysis, and visualization.

3) Integration of Hardware and Software: As shown in Fig. 12, the signals of the vibration sensor and current sensor on V40 machine are collected by WebDAQ machine and send to edge devices through industrial personal computer (IPC) and TSN switch. End devices and edge devices in the same network segment (192.168.0) can communicate with each other, but are

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

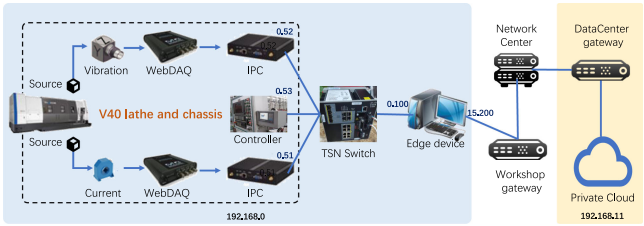


Fig. 12. Infrastructure architecture of real-time predictive maintenance of cutting tools.

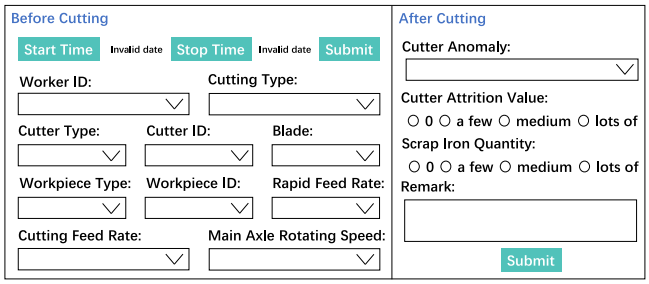


Fig. 13. Labeling system of real-time predictive maintenance of cutting tools.

not able to communicate directly with other network segments (e.g., 192.168.11) of the private cloud. To solve this problem, we configured two network cards inside the edge node and shared them through the network bridge, thus realizing data sharing between workshops and the private cloud. The integration of IndustEdge with the infrastructure above is the same as in the first case study.

4) Evaluation Metrics: We collected real cutter status time series data in the workshop of the aviation engine manufactory for the performance evaluation. Specifically, we installed high-frequency vibration signal sensor and current signal sensor on the V40 machine. The vibration signals and current signals were sent to the edge device. Furthermore, we developed the labeling system as shown in Fig. 13 to record the state of the cutter and iron chips (if too many iron chips are produced in the process and not remove, they will affect the workpiece processing and cutter attrition). Finally, we collected the labelled data of one month with the total size of about 1138 G. To demonstrate the effectiveness and efficiency of IndustEdge, we used the accuracy for machine running state prediction, cutter remaining useful life prediction and chip iron quantity prediction applications, the intraclass to interclass distance for cutter anomaly detection, the response latency for four applications above and the controlling machine application.

5) Experiment Results: The accuracy of the machine running state prediction, cutter remaining useful life prediction and chip iron quantity prediction reach up to 98%, 99%, and 85.2%, respectively. The intraclass to interclass distance of cutter anomaly detection is 0.68. Furthermore, the response latency of the four applications is less than 0.3 s, and the response latency of the controlling machine is 10 ms after detected an abnormal cutter.

VI. Conclusion

We present IndustEdge in this article, which was an ECCI platform for smart industry. On the one hand, it took TSN as deterministic transport for the network domain to provide guaranteed data transport with bounded low latency for time-critical traffic of the industrial operation application. On the other hand, it provided an extensible orchestration component to develop various orchestration algorithms based on resource constraints and TSN characteristics to reduce the system level latency. It could also provide the life cycle management of infrastructure and ECCI applications by the Docker-based operation and maintenance system, and the transparent communication of ECCI applications to users could be realized by automatic routing. Moreover, IndustEdge provided a rich library of ECCI algorithms and use cases maintained by the image repository for the development of ECCI applications. We implemented the IndustEdge prototype and emulation platforms for real and emulated industrial scenarios, respectively. Through two thorough case studies, extensive experimental results demonstrated both the effectiveness and efficiency of IndustEdge. In our future work, we aim to concentrate on the orchestration algorithms that fully combines the characteristics of microservices and TSN, so as to ensure the latency requirements of applications in the typical industrial scenarios.

References

[1] G. Aceto, V. Persico, and A. Pescapé, "A survey on information and communication technologies for industry 4.0: State-of-the-art, taxonomies, perspectives, and challenges," *IEEE Commun. Surv. Tut.*, vol. 21, no. 4, pp. 3467–3501, Oct.–Dec. 2019.

[2] M. Aazam, S. Zeadally, and K. A. Harras, "Deploying fog computing in industrial internet of things and industry 4.0," *IEEE Trans. Ind. Informat.*, vol. 14, no. 10, pp. 4674–4682, Oct. 2018.

[3] D. Wu *et al.*, "Cloud manufacturing: Strategic vision and state-of-the-art," *J. Manuf. Syst.*, vol. 32, no. 4, pp. 564–579, 2013.

[4] X. V. Wang and L. Wang, "From cloud manufacturing to cloud remanufacturing: A cloud-based approach for WEEE recovery," *Manuf. Lett.*, vol. 2, no. 4, pp. 91–95, 2014.

[5] P. Yang, F. Lyu, W. Wu, N. Zhang, L. Yu, and X. Shen, "Edge coordinated query configuration for low-latency and accurate video analytics," *IEEE Trans. Ind. Informat.*, vol. 16, no. 7, pp. 4855–4864, Jul. 2020.

[6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.

[7] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE Inst. Electr. Electron. Eng.*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.

[8] Y. Wang, K. Hong, J. Zou, T. Peng, and H. Yang, "A CNN-based visual sorting system with cloud-edge computing for flexible manufacturing systems," *IEEE Trans. Ind. Informat.*, vol. 16, no. 7, pp. 4726–4735, Jul. 2020.

[9] X. V. Wang and X. W. Xu, "An interoperable solution for cloud manufacturing," *Robot. Comput.-Integr. Manuf.*, vol. 29, no. 4, pp. 232–247, 2013.

[10] Y. Lu, X. Xu, and J. Xu, "Development of a hybrid manufacturing cloud," *J. Manuf. Syst.*, vol. 33, no. 4, pp. 551–566, 2014.

[11] Z. Sang and X. Xu, "The framework of a cloud-based CNC system," *Procedia Cirp*, vol. 63, pp. 82–88, 2017.

[12] Microsoft Azure, *Azure IOT Edge*, 2018. [Online]. Available: https://azure.microsoft.com/en-us/services/iot-edge/

[13] Alibaba cloud, *Link Edge*, 2018. [Online]. Available: https://iot.aliyun.com/products/linkedge/

[14] H. Gupta *et al.*, "iFogSim: A. toolkit for modeling and simulation of resource management techniques in the Internet of Things, edge and fog computing environments," *Softw. Pract. Exper.*, vol. 47, no. 9, pp. 1275–1296, 2017.

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.

[15] T. Qayyum, A. W. Malik, and M. Khattak, "FogNetSim++: A toolkit for modeling and simulation of distributed fog environment," *IEEE Access*, vol. 6, pp. 63570–63583, 2018.
[16] A. Brogi and S. Forti, "Qos-aware deployment of IoT applications through the fog," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1185–1192, Oct. 2017.
[17] R. Mayer, L. Graser, H. Gupta, E. Saurez, and U. Ramachandran, "Emufog: Extensible and scalable emulation of large-scale fog computing infrastructures," in *Proc. IEEE Fog World Congress*, 2017, pp. 1–6.
[18] A. Coutinho, F. Greve, C. Prazeres, and J. Cardoso, "Fogbed: A rapid-prototyping emulation environment for fog computing," in *Proc. IEEE Int. Conf. Commun.*, 2018, pp. 1–7.
[19] J. Hasenburg, M. Grambow, E. Grünewald, S. Huk, and D. Bermbach, "MockFog: Emulating fog computing infrastructure in the cloud," in *Proc. of IEEE Int. Conf. Fog Comput.*, 2019, pp. 144–152.
[20] W. Steiner, S. S. Craciunas, and R. S. Oliver, "Traffic planning for time-sensitive communication," *IEEE Commun. Standards Mag.*, vol. 2, no. 2, pp. 42–47, Jun. 2018.
[21] A. A. Atallah, G. B. Hamad, and O. A. Mohamed, "Routing and scheduling of time-triggered traffic in time-sensitive networks," *IEEE Trans. Ind. Informat.*, vol. 16, no. 7, pp. 4525–4534, Jul. 2020.
[22] A. Varga and R. Hornig, "An overview of the Omnet simulation environment," in *Proc. EAI SIMUtools*, 2008, p. 1–10.
[23] P. Heise, F. Geyer, and R. Obermaisser, "TSimNet: An industrial time sensitive networking simulation framework based on Omnet," in *Proc. IEEE 8th IFIP Int. Conf. New Technol., Mobility Secur.*, 2016, pp. 1–5.
[24] M. Pahlevan and R. Obermaisser, "Evaluation of time-triggered traffic in time-sensitive networks using the Opnet simulation framework," in *Proc. IEEE 26th Euromicro Int. Conf. Parallel, Distrib. Netw.-Based Process.*, 2018, pp. 283–287.
[25] J. Falk *et al.*, "NeSTiNg: Simulating ieee time-sensitive networking (TSN) in Omnet," in *Proc. IEEE Int. Conf. Networked Syst.*, 2019, pp. 1–8.
[26] Y. Wang *et al.*, "MPCSM: Microservice placement for edge-cloud collaborative smart manufacturing," *IEEE Trans. Ind. Informat.*, vol. 17, no. 9, pp. 5898–5908, Sep. 2021.
[27] T. Zhang and B. Liu, "Exposing end-to-end delay in software-defined networking," *Int. J. Reconfigurable Comput.*, vol. 2019, pp. 1–13, 2019.
[28] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux containers," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2015, pp. 171–172.
[29] S. Wang, S. Yang, and C. Zhao, "Surveiledge: Real-time video query based on collaborative cloud-edge deep learning," in *Proc. IEEE INFOCOM-IEEE Conf. Comput. Commun.*, 2020, pp. 2519–2528.



Yimeng Wang received the M.Sc. degree in 2018 from Xi'an Jiaotong University (XJTU), Xi'an, China, where she is currently working toward the Ph.D. degree with the School of Computer Science and Technology, both computer science and technology.

Her research interests include edge computing, industrial intelligence, and system optimization.



Shusen Yang (Senior Member, IEEE) received the Ph.D. degree in computing from Imperial College London, London, U.K., in 2014.

He is currently a Professor and Director of the National Engineering Laboratory for Big Data Analytics, and Deputy Director of Ministry of Education (MoE) Key Lab for Intelligent Networks and Network Security, both with Xi'an Jiaotong University (XJTU), Xi'an, China. He is also the Deputy Director of the Industrial Artificial Intelligence Center, Pazhou Laboratory, Guangzhou, China. Before joining XJTU, he worked as a Lecturer (Assistant Professor) with the University of Liverpool, Liverpool, U.K., from 2015 to 2016, and a Research Associate with Intel Collaborative Research Institute (ICRI) on sustainable connected cities from 2013 to 2014. His research interests include distributed systems and data sciences, and their applications in industrial scenarios, including data-driven network algorithms, distributed machine learning, Edge-Cloud intelligence, industrial Internet, and industrial intelligence.

Dr. Shusen is a DAMO Academy Young Fellow, and an Honorary Research Fellow with Imperial College London, London. He is a Member of ACM.



Xuebin Ren (Member, IEEE) received the Ph.D. degree in computer science and technology from the Department of Computer Science and Technology, Xi'an Jiaotong University, Xi'an, China, in 2017.

He is currently an Associate Professor with Xi'an Jiaotong University and a Member of National Engineering Laboratory for Big Data Analytics (NEL-BDA), Beijing, China. He has been a Visiting Ph.D. Student with the Department of Computing, Imperial College London, London, U.K., from 2016 to 2017. His research interests include data privacy protection, federated learning, and privacy-preserving machine learning.

Dr. Ren is a Member of the ACM.



Peng Zhao received the B.S. and Ph.D. degrees in computer science and technology from Xi'an Jiaotong University, Xi'an, China, in 2007 and 2013, respectively.

He is currently an Associate Professor with the School of Computer Science and Technology, Xi'an Jiaotong University. His research interests include distributed systems, cloud computing, and system optimization.



Cong Zhao received the Ph.D. degree in computer science and technology from Xi'an Jiaotong University (XJTU), Xi'an, China, in 2017.

He is currently a Research Associate with the Department of Computing, Imperial College London, London, U.K. His research interests include edge computing, meta learning, computing economics, and people-centric sensing.



Xinyu Yang received the B.S., M.S., and Ph.D. degrees in computer science and technology from Xi'an Jiaotong University (XJTU), Xi'an, China, in 1995, 1997, and 2001, respectively.

He is currently a Professor with the Department of Computer Science and Technology, XJTU. His research interests include wireless communication, mobile ad hoc networks, and network security and privacy.

Authorized licensed use limited to: UNIVERSIDADE DE PERNAMBUO. Downloaded on November 19,2025 at 14:59:32 UTC from IEEE Xplore. Restrictions apply.